

Jupyter Notebook Execution Report

Name: Hany El Atlassi

Date: September 10, 2026

Cell 1: ■ Markdown

TP2: Text Mining

Hany El Atlassi

Cell 2: ■ Markdown

Étape 1 - Importer Scikit-learn

Cell 3: ■ Code

```
import sklearn
from sklearn.datasets import fetch_20newsgroups
```

Cell 4: ■ Markdown

Étape 2 - Importer l'ensemble de données de Scikit-learn

Cell 5: ■ Code

```
data = fetch_20newsgroups()
print(type(data))
```

Output:

```
<class 'sklearn.utils._bunch.Bunch'>
```

Cell 6: ■ Code

```
print(dir(data))
```

Output:

```
['DESCR', 'data', 'filenames', 'target', 'target_names']
```

Cell 7: ■ Code

```
data.target
```

Output:

```
array([7, 4, 4, ..., 3, 1, 8])
```

Cell 8: ■ Code

```
data.target_names
```

Output:

```
['alt.atheism', 'comp.graphics', 'comp.os.ms-windows.misc', 'comp.sys.ibm.pc.hardware', 'comp.sys
```

Cell 9: ■ Code

```
data.data[:2]
```

Output:

```
["From: lerxst@wam.umd.edu (where's my thing)\nSubject: WHAT car is this!?\nNntp-Posting-Host: ra
```

Cell 10: ■ Code

```
for i in range(20):  
    print("Classe: %s, Nombre textes: %s" % (i, len(data.target[data.target==i])))
```

Output:

```
Classe: 0, Nombre textes: 480  
Classe: 1, Nombre textes: 584  
Classe: 2, Nombre textes: 591  
Classe: 3, Nombre textes: 590  
Classe: 4, Nombre textes: 578  
Classe: 5, Nombre textes: 593  
Classe: 6, Nombre textes: 585  
Classe: 7, Nombre textes: 594  
Classe: 8, Nombre textes: 598  
Classe: 9, Nombre textes: 597
```

```
Classe: 10, Nombre textes: 600
Classe: 11, Nombre textes: 595
Classe: 12, Nombre textes: 591
Classe: 13, Nombre textes: 594
Classe: 14, Nombre textes: 593
Classe: 15, Nombre textes: 599
Classe: 16, Nombre textes: 546
Classe: 17, Nombre textes: 564
Classe: 18, Nombre textes: 465
Classe: 19, Nombre textes: 377
```

Cell 11: ■ Code

```
len(data.target)
```

Output:

```
11314
```

Cell 12: ■ Code

```
categories = ["comp.os.ms-windows.misc", "comp.sys.ibm.pc.hardware",
"comp.sys.mac.hardware", "comp.windows.x"]
```

Cell 13: ■ Code

```
dataset = fetch_20newsgroups(subset='train', categories=categories, shuffle=True,
random_state=42)
```

Cell 14: ■ Code

```
dataset.target_names
```

Output:

```
['comp.os.ms-windows.misc', 'comp.sys.ibm.pc.hardware', 'comp.sys.mac.hardware', 'comp.windows.x']
```

Cell 15: ■ Code

```
for i in range(4):
    print("Classe: %s, Nombre textes: %s" % (i, len(data.target[data.target==i])))
```

Output:

```
Classe: 0, Nombre textes: 480
Classe: 1, Nombre textes: 584
Classe: 2, Nombre textes: 591
Classe: 3, Nombre textes: 590
```

Cell 16: ■ Markdown

Étape 3 - Organisation de données

Cell 17: ■ Code

```
from sklearn.feature_extraction.text import CountVectorizer
count_vect = CountVectorizer()
x_train_counts = count_vect.fit_transform(dataset.data)
x_train_counts.shape
```

Output:

```
(2352, 60987)
```

Cell 18: ■ Code

```
count_vect.vocabulary_.get(u'os')
```

Output:

```
42292
```

Cell 19: ■ Code

```
count_vect.vocabulary_.get(u'mac')
```

Output:

```
36430
```

Cell 20: ■ Code

```
from sklearn.feature_extraction.text import TfidfTransformer
tf_transformer = TfidfTransformer(use_idf=False).fit(x_train_counts)
```

```
x_train_tf = tf_transformer.transform(x_train_counts)
x_train_tf.shape
```

Output:

```
(2352, 60987)
```

Cell 21: ■ Code

```
tfidf_transformer = TfidfTransformer()
x_train_tfidf = tfidf_transformer.fit_transform(x_train_counts)
x_train_tfidf.shape
```

Output:

```
(2352, 60987)
```

Cell 22: ■ Markdown

Étape 4 - Construire le modèle

Cell 23: ■ Code

```
from sklearn.naive_bayes import MultinomialNB
clf = MultinomialNB().fit(x_train_tfidf, dataset.target)
```

Cell 24: ■ Code

```
docs_new = ["Mac est une machine qui utilise le system d'exploitation mac os",
"OpenGL on the GPU is fast"]
x_new_counts = count_vect.transform(docs_new)
x_new_tfidf = tfidf_transformer.transform(x_new_counts)

predicted = clf.predict(x_new_tfidf)

for doc, category in zip(docs_new, predicted):
    print("%r => %s"%(doc, dataset.target_names[category]))
```

Output:

```
"Mac est une machine qui utilise le system d'exploitation mac os" => comp.sys.mac.hardware
'OpenGL on the GPU is fast' => comp.sys.ibm.pc.hardware
```

Cell 25: ■ Code

```
from sklearn.pipeline import Pipeline

text_clf = Pipeline([('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),
                     ('clf', MultinomialNB())])
```

Cell 26: ■ Code

```
text_clf.fit(dataset.data, dataset.target)
```

Output:

```
Pipeline(steps=[('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),
                 ('clf', MultinomialNB())])
```

Cell 27: ■ Markdown

Étape 5 - Évaluation du modèle et de sa précision

Cell 28: ■ Code

```
import numpy as np

text_clf.fit(dataset.data, dataset.target)

dataset_test = fetch_20newsgroups(subset='test', categories=categories,
                                  shuffle=True, random_state=42)

docs_test = dataset_test.data

predicted = text_clf.predict(docs_test)

np.mean(predicted==dataset_test.target)
```

Output:

```
np.float64(0.834610472541507)
```

Cell 29: ■ Code

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(dataset_test.target, predicted,
                             target_names=dataset_test.target_names))
```

Output:

```
precision    recall  f1-score   support
```

comp.os.ms-windows.misc	0.84	0.73	0.78	394
comp.sys.ibm.pc.hardware	0.72	0.89	0.80	392
comp.sys.mac.hardware	0.89	0.86	0.87	385
comp.windows.x	0.93	0.85	0.89	395
accuracy			0.83	1566
macro avg	0.84	0.83	0.84	1566
weighted avg	0.84	0.83	0.84	1566

Cell 30: ■ Code

```
print(confusion_matrix(dataset_test.target, predicted))
```

Output:

```
[[289  71  15  19]
 [ 16 350  22   4]
 [ 13  39 331   2]
 [ 27  25   6 337]]
```

Cell 31: ■ Markdown

Exercise

Cell 32: ■ Code

```
from sklearn import svm

svm_model = svm.SVC()
```

Cell 33: ■ Code

```
text_svm = Pipeline([('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),
                     ('svm', svm_model)])

text_svm.fit(dataset.data, dataset.target)
```

Output:

```
Pipeline(steps=[('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),
                 ('svm', SVC())])
```

Cell 34: ■ Code

```
svm_predicted = text_svm.predict(docs_test)
```

Cell 35: ■ Code

```
print(classification_report(dataset_test.target, svm_predicted,  
target_names=dataset_test.target_names))
```

Output:

	precision	recall	f1-score	support
comp.os.ms-windows.misc	0.81	0.78	0.79	394
comp.sys.ibm.pc.hardware	0.77	0.85	0.81	392
comp.sys.mac.hardware	0.88	0.89	0.88	385
comp.windows.x	0.93	0.86	0.89	395
accuracy			0.84	1566
macro avg	0.85	0.84	0.84	1566
weighted avg	0.85	0.84	0.84	1566

Cell 36: ■ Code

```
print(confusion_matrix(dataset_test.target, svm_predicted))
```

Output:

```
[[306  58  15  15]  
 [ 24 334  27   7]  
 [   5  32 343   5]  
 [ 41  10   6 338]]
```

Cell 37: ■ Code

```
from sklearn.neighbors import KNeighborsClassifier  
  
knn_model = KNeighborsClassifier(n_neighbors=3)  
text_knn = Pipeline([('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),  
('svm', knn_model)])  
text_knn.fit(dataset.data, dataset.target)
```

Output:

```
Pipeline(steps=[('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),
                 ('svm', KNeighborsClassifier(n_neighbors=3))])
```

Cell 38: ■ Code

```
knn_predicted = text_knn.predict(docs_test)
```

Cell 39: ■ Code

```
print(classification_report(dataset_test.target, knn_predicted,
                             target_names=dataset_test.target_names))
```

Output:

	precision	recall	f1-score	support
comp.os.ms-windows.misc	0.59	0.62	0.61	394
comp.sys.ibm.pc.hardware	0.68	0.67	0.68	392
comp.sys.mac.hardware	0.77	0.64	0.70	385
comp.windows.x	0.66	0.76	0.71	395
accuracy			0.67	1566
macro avg	0.68	0.67	0.67	1566
weighted avg	0.68	0.67	0.67	1566

Cell 40: ■ Code

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier()
text_dt = Pipeline([('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),
                    ('svm', dt_model)])
text_dt.fit(dataset.data, dataset.target)
```

Output:

```
Pipeline(steps=[('vect', CountVectorizer()), ('tfidf', TfidfTransformer()),
                 ('svm', DecisionTreeClassifier())])
```

Cell 41: ■ Code

```
dt_predicted = text_dt.predict(docs_test)
print(classification_report(dataset_test.target, dt_predicted,
target_names=dataset_test.target_names))
```

Output:

	precision	recall	f1-score	support
comp.os.ms-windows.misc	0.63	0.73	0.68	394
comp.sys.ibm.pc.hardware	0.67	0.61	0.64	392
comp.sys.mac.hardware	0.77	0.79	0.78	385
comp.windows.x	0.68	0.62	0.65	395
accuracy			0.69	1566
macro avg	0.69	0.69	0.69	1566
weighted avg	0.69	0.69	0.69	1566

Cell 42: ■ Code

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import pandas as pd
```

```
results = {
    "Naïve Bayes": predicted,
    "SVM": svm_predicted,
    "KNN": knn_predicted,
    "Decision Tree": dt_predicted,
}
```

```
comparison = []
for name, preds in results.items():
    accuracy = accuracy_score(dataset_test.target, preds)
    precision, recall, f1, _ = precision_recall_fscore_support(dataset_test.target,
    preds, average="weighted")
    comparison.append({"Méthode": name, "Accuracy": accuracy, "Precision": precision,
    "Recall": recall, "F1-score": f1})
```

```
comparison_df = pd.DataFrame(comparison).set_index("Méthode")
comparison_df
```

Output:

	Accuracy	Precision	Recall	F1-score
Méthode				
Naive Bayes	0.834610	0.843799	0.834610	0.835591
SVM	0.843550	0.846644	0.843550	0.844100
KNN	0.672414	0.677637	0.672414	0.672636
Decision Tree	0.687101	0.687892	0.687101	0.686025

Cell 43: ■ Code

```
import matplotlib.pyplot as plt

ax = comparison_df.T.plot(kind="bar", figsize=(9, 5), rot=0)
plt.ylabel("Score")
plt.title("Comparaison des 4 méthodes de classification")
plt.ylim(0.6, 1.0)
plt.legend(title="Méthode", loc="lower right")

for container in ax.containers:
    ax.bar_label(container, fmt="%.2f", fontsize=8, padding=2)

plt.tight_layout()
plt.show()
```

Output:

[STDERR]

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

